



Plano Técnico Completo — Agente de IA Local

Estilo Manus

Versão 2.1 — Linux Mint + Google Chrome + Banco de Dados no HD de Backup

Sistema Operacional: Linux Mint (Ubuntu/Debian base)

Navegador: Google Chrome já instalado

Objetivo: Construir um agente de IA autônomo, acessível pela internet, rodando em PC local com hardware modesto, usando DeepSeek como cérebro principal, Playwright com Chrome para controle de navegador, e Cloudflare Tunnel para exposição segura com HTTPS.



Índice

1. [Visão Geral do Sistema](#)
2. [Por que Linux Mint é uma Vantagem](#)
3. [Arquitetura Completa](#)
4. [Hardware e Viabilidade](#)
5. [Estrutura de Pastas do Projeto](#)
6. [Backend — FastAPI + Orquestrador](#)
7. [Sistema de Ferramentas \(Tools\)](#)
8. [Integração com IA — DeepSeek + Visão](#)
9. [Frontend — Interface Web](#)
10. [WebSocket — Logs em Tempo Real](#)
11. [Segurança e Autenticação](#)
12. [Cloudflare Tunnel — Exposição Externa](#)
13. [Banco de Dados — SQLite](#)

14. [Rodando como Serviço do Sistema \(systemd\)](#)
 15. [Fluxo Completo de Execução](#)
 16. [Guia de Instalação Passo a Passo — Linux Mint](#)
 17. [Configuração do Ambiente \(.env\)](#)
 18. [Comandos do Dia a Dia](#)
 19. [Roadmap de Implementação](#)
 20. [Limitações e Riscos](#)
-

1. Visão Geral do Sistema

O sistema é um **agente de IA autônomo** que recebe uma tarefa em linguagem natural e a executa de forma autônoma usando ferramentas reais: navegador, terminal, arquivos e captura de tela. É inspirado no Manus AI, porém adaptado para rodar em hardware local modesto com Linux Mint.

O que o agente consegue fazer

- Receber uma tarefa como: *"Pesquise os 5 melhores frameworks Python de 2024 e salve um relatório em PDF"*
 - Planejar os passos necessários usando a DeepSeek API
 - Controlar o Google Chrome já instalado no sistema via Playwright (sem baixar outro navegador)
 - Executar comandos de terminal Linux seguros via whitelist
 - Tirar prints da tela com MSS e analisá-los com IA de visão
 - Salvar arquivos gerados em uma pasta isolada (`~/manus-workspace/`)
 - Transmitir logs em tempo real para o usuário via WebSocket
 - Pedir confirmação do usuário antes de ações críticas
 - Ser acessado de qualquer lugar pelo domínio próprio com HTTPS via Cloudflare Tunnel
 - Iniciar automaticamente com o sistema via **systemd** (sem precisar abrir terminal)
-

2. Por que Linux Mint é uma Vantagem

Linux Mint transforma esse projeto em algo **mais simples, mais rápido e mais estável** do que seria no Windows. Aqui está o motivo:

Vantagens Diretas

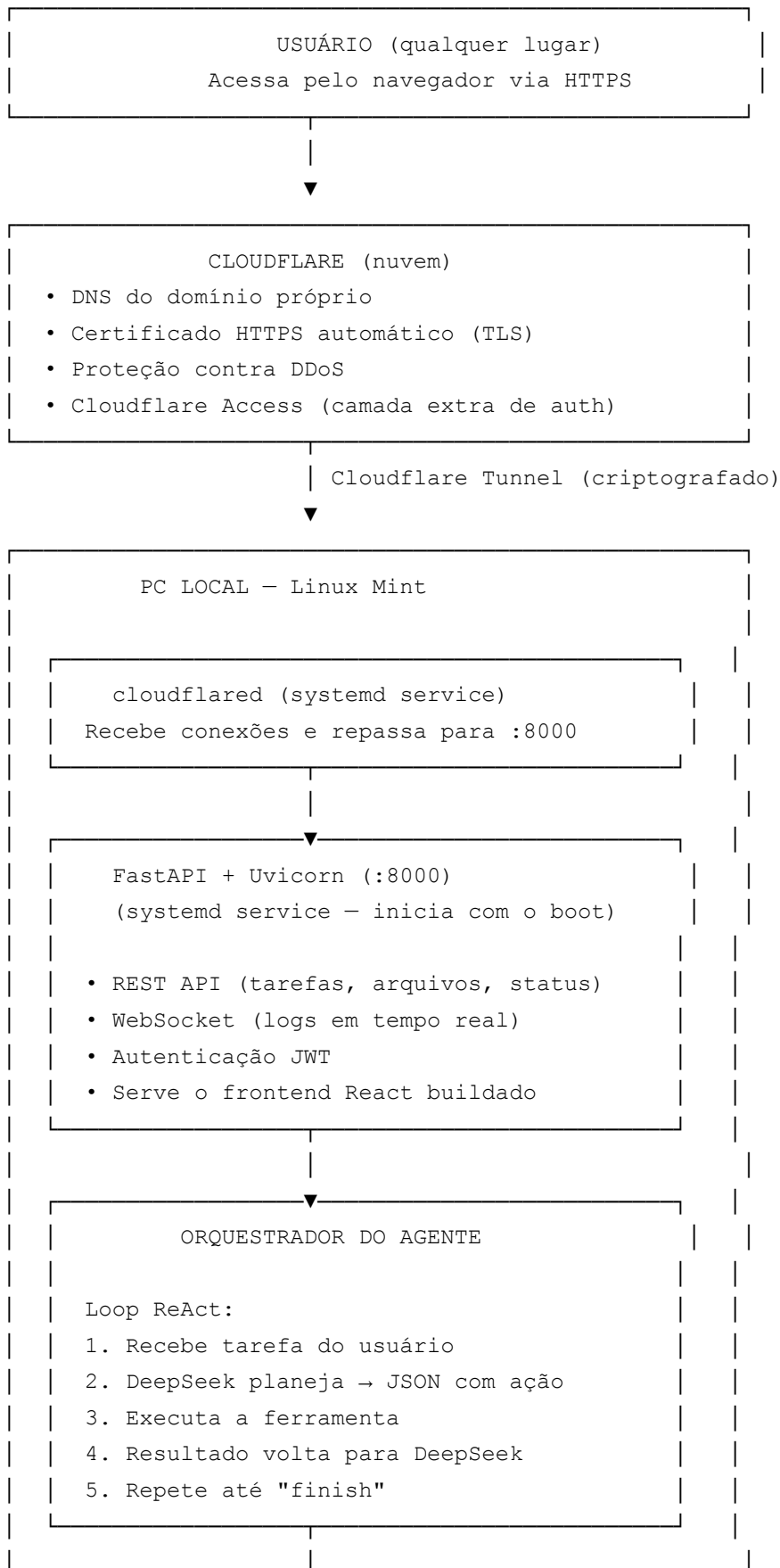
Aspecto	Windows	Linux Mint 
Python 3.11+	Requer instalação manual, PATH confuso	Já disponível via <code>apt</code> , <code>venv</code> funciona perfeitamente
Processos em background	Requer Task Scheduler ou NSSM	systemd nativo — simples e robusto
Playwright com Chrome	Requer configuração extra de caminhos	Detecta Chrome do sistema automaticamente
Permissões de arquivo	UAC, permissões complexas	Modelo Unix direto e previsível
Terminal	PowerShell/CMD com sintaxe diferente	bash — todos os exemplos deste plano funcionam direto
MSS (screenshots)	Funciona, mas pode ter glitches com DPI	Funciona perfeitamente com X11
PyAutoGUI	Requer dependências extras	Funciona nativamente com Xlib
Cloudflared	Instalação como serviço complexa	<code>systemd</code> torna isso trivial
Consumo de RAM do SO	~3-4 GB só do Windows	~800 MB - 1.2 GB do Mint (sobra muito mais para o agente)

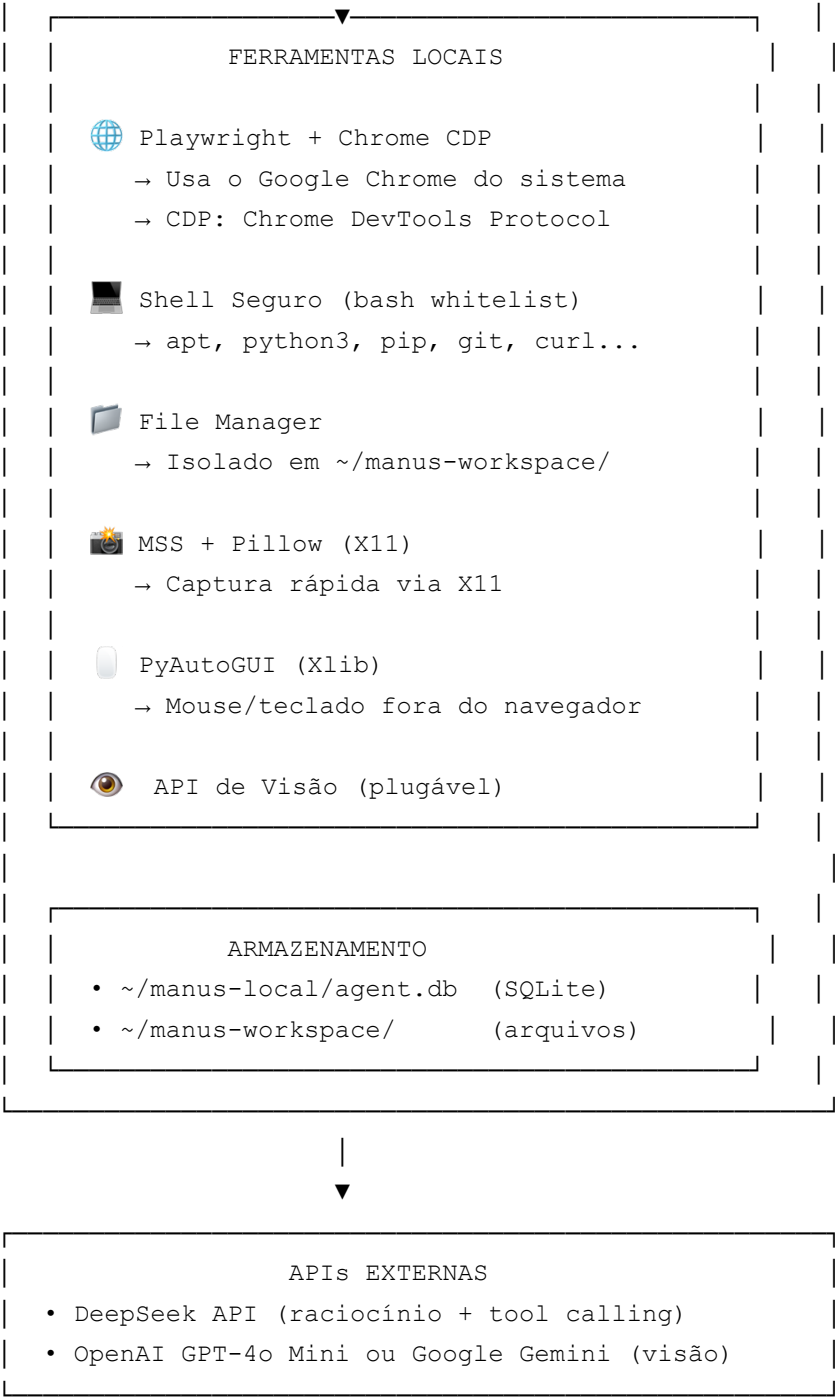
Linux Mint deixa mais RAM disponível para o agente

```
Windows 10/11:    16 GB total - ~3.5 GB (SO) = ~12.5 GB disponível
Linux Mint:       16 GB total - ~1.0 GB (SO) = ~15 GB disponível
```

Isso significa que o Playwright rodando Chrome, o FastAPI e os outros processos têm mais espaço para respirar.

3. Arquitetura Completa





4. Hardware e Viabilidade

Seu Hardware no Linux Mint

Componente	Especificação	Papel no Sistema
CPU	i5 antigo	Executa Python, FastAPI, Chrome via Playwright

RAM	16 GB DDR	Excelente. Mint usa ~1 GB, sobram ~15 GB
GPU	Integrada	Não usada — IA fica nas APIs
SO	Linux Mint	✅ Ideal — leve, estável, Unix nativo
Navegador	Google Chrome	✅ Playwright conecta via CDP sem instalar nada
Internet	Necessária	Tunnel + APIs de IA

Estimativa de Uso de Recursos

Processo	RAM estimada	CPU estimada
Linux Mint (SO + GUI)	~800 MB - 1.2 GB	Mínimo em idle
FastAPI + Uvicorn	~150 MB	Baixo
Chrome controlado pelo Playwright	~400-700 MB	Médio durante navegação
MSS + Pillow (screenshots)	~50 MB	Baixo, picos rápidos
cloudflared	~30 MB	Muito baixo
SQLite	~20 MB	Mínimo
Total	~1.5 GB - 2.3 GB	Médio

Com 16 GB disponíveis, você tem mais de 13 GB de folga. Até múltiplas tarefas simultâneas seriam viáveis futuramente.

Vantagem do Chrome via CDP

Playwright no Linux pode **conectar ao Google Chrome já instalado** usando o protocolo CDP (Chrome DevTools Protocol), sem precisar baixar o Chromium separado. Isso economiza ~400 MB em disco e garante que o agente use exatamente o mesmo Chrome que você usa no dia a dia.

5. Estrutura de Pastas do Projeto

```
~/manus-local/                                # Diretório principal do projeto
|
└─ backend/
```

```

|   ├── main.py                # Ponto de entrada FastAPI
|   ├── config.py              # Lê o .env com pydantic-settings
|   ├── auth.py                # Autenticação JWT
|   ├── database.py            # SQLite assíncrono
|   |
|   ├── agent/
|   |   ├── orchestrator.py    # Loop principal ReAct
|   |   ├── planner.py         # Comunicação com DeepSeek
|   |   ├── tool_executor.py   # Roteador de ferramentas
|   |   └── state.py           # Enum de status do agente
|   |
|   ├── tools/
|   |   ├── __init__.py
|   |   ├── browser.py        # Playwright + Chrome CDP
|   |   ├── shell.py          # bash com whitelist
|   |   ├── file_manager.py    # Leitura/escrita isolada
|   |   ├── screenshot.py      # MSS via X11
|   |   ├── pyautogui_tool.py  # Mouse/teclado (Xlib)
|   |   └── vision.py          # API de visão plugável
|   |
|   ├── api/
|   |   ├── tasks.py           # CRUD de tarefas
|   |   ├── files.py           # Download de arquivos gerados
|   |   ├── ws.py              # WebSocket broadcaster
|   |   └── control.py         # pause / stop / confirm
|   |
|   └── requirements.txt
|
|── frontend/
|   ├── src/
|   |   ├── App.jsx
|   |   ├── components/
|   |   |   ├── ChatPanel.jsx
|   |   |   ├── LogPanel.jsx
|   |   |   ├── ScreenView.jsx
|   |   |   ├── FileList.jsx
|   |   |   ├── StatusBadge.jsx
|   |   |   └── ConfirmDialog.jsx
|   |   └── hooks/
|   |       └── useWebSocket.js
|   ├── package.json
|   └── vite.config.js
|
|── systemd/                    # Arquivos de serviço Linux
|   ├── manus-agent.service    # Serviço do FastAPI
|   └── manus-tunnel.service    # Serviço do cloudflared

```

```

├─ scripts/
│   ├─ install.sh          # Script de instalação completa
│   ├─ start.sh           # Inicia tudo manualmente
│   └─ stop.sh            # Para tudo
│
├─ .env                   # Variáveis de ambiente (não commitar)
├─ .env.example
├─ cloudflared-config.yml
└─ README.md

~/manus-workspace/      # PASTA ISOLADA do agente (fora do projeto)

```

Nota de segurança: O workspace fica em `~/manus-workspace/`, separado do código do projeto. O agente nunca consegue escrever fora dessa pasta.

6. Backend — FastAPI + Orquestrador

backend/main.py

```

from fastapi import FastAPI, Depends
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager

from database import init_db
from api import tasks, files, ws, control
from auth import auth_router, verify_token

@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    yield

app = FastAPI(title="Manus Local Agent", version="2.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth_router, prefix="/auth", tags=["auth"])

```



```

app.include_router(tasks.router, prefix="/api/tasks", tags=["tasks"],
                    dependencies=[Depends(verify_token)])
app.include_router(files.router, prefix="/api/files", tags=["files"],
                    dependencies=[Depends(verify_token)])
app.include_router(control.router, prefix="/api/control", tags=["control"],
                    dependencies=[Depends(verify_token)])
app.include_router(ws.router) # WebSocket tem auth própria via query param

# Frontend React buildado servido como estático
app.mount("/", StaticFiles(directory="../frontend/dist", html=True), name="fronte

```

backend/config.py

```

from pydantic_settings import BaseSettings
from pathlib import Path

class Settings(BaseSettings):
    # Segurança
    SECRET_KEY: str
    ADMIN_USER: str
    ADMIN_PASS: str

    # APIs
    DEEPSEEK_API_KEY: str
    VISION_PROVIDER: str = "openai"
    OPENAI_API_KEY: str = ""
    GOOGLE_API_KEY: str = ""

    # Agente
    MAX_ITERATIONS: int = 30
    WORKSPACE_PATH: Path = Path.home() / "manus-workspace"

    # Chrome - caminho padrão no Linux
    CHROME_BINARY: str = "/usr/bin/google-chrome"

    # Cloudflare
    EXTERNAL_DOMAIN: str = ""

    class Config:
        env_file = ".env"

settings = Settings()

# Garante que a workspace existe
settings.WORKSPACE_PATH.mkdir(parents=True, exist_ok=True)

```

backend/agent/orchestrator.py

```
import asyncio
import json
from agent.planner import DeepSeekPlanner
from agent.state import AgentState, AgentStatus
from tools.tool_executor import execute_tool
from database import save_log

class AgentOrchestrator:
    def __init__(self, task_id: str, task: str, ws_broadcaster):
        self.task_id = task_id
        self.task = task
        self.broadcast = ws_broadcaster
        self.planner = DeepSeekPlanner()
        self.state = AgentState(task_id=task_id)
        self.conversation_history = []
        self.max_iterations = 30
        self._paused = False
        self._stopped = False
        self._confirmation_event = asyncio.Event()
        self._confirmation_approved = True

    async def run(self):
        await self.log("🚀 Agente iniciado no Linux Mint", "info")
        await self.set_status(AgentStatus.THINKING)

        self.conversation_history.append({
            "role": "user",
            "content": f"Tarefa: {self.task}"
        })

        for iteration in range(self.max_iterations):
            if self._stopped:
                await self.log("🛑 Execução interrompida", "warning")
                await self.set_status(AgentStatus.STOPPED)
                break

            while self._paused and not self._stopped:
                await asyncio.sleep(0.5)

            await self.log(f"🧠 Iteração {iteration + 1} - consultando DeepSeek..", "info")
            await self.set_status(AgentStatus.THINKING)

            try:
                response = await self.planner.get_next_action(self.conversation_h
```

```

except Exception as e:
    await self.log(f"❌ Erro ao consultar DeepSeek: {e}", "error")
    await self.set_status(AgentStatus.ERROR)
    break

self.conversation_history.append({
    "role": "assistant",
    "content": json.dumps(response, ensure_ascii=False)
})

action = response.get("action")

if action == "finish":
    await self.log(f"✅ Tarefa concluída: {response.get('result', '')}", "success")
    await self.set_status(AgentStatus.DONE)
    break

if response.get("requires_confirmation"):
    approved = await self.request_confirmation(
        response.get("confirmation_message", "Confirmar esta ação?")
    )
    if not approved:
        await self.log(f"❌ Ação cancelada pelo usuário", "warning")
        self.conversation_history.append({
            "role": "user",
            "content": "O usuário cancelou esta ação. Reavalie o plan"
        })
        continue

desc = response.get("description", "")
await self.log(f"🔧 {action}: {desc}", "action")
await self.set_status(AgentStatus.EXECUTING)

tool_result = await execute_tool(
    tool_name=action,
    params=response.get("params", {}),
    task_id=self.task_id
)

# Envia screenshot automaticamente após ações no navegador
if action.startswith("browser_") and "screenshot" not in action:
    await self._send_auto_screenshot()

result_preview = str(tool_result)[:400]
await self.log(f"📄 Resultado: {result_preview}", "result")

self.conversation_history.append({

```

```

        "role": "user",
        "content": f"Resultado de '{action}': {json.dumps(tool_result, en
    }}

    else:
        await self.log("⚠ Limite máximo de iterações atingido", "warning")
        await self.set_status(AgentStatus.ERROR)

async def _send_auto_screenshot(self):
    """Captura e envia screenshot do Chrome para o frontend."""
    try:
        from tools.screenshot import ScreenshotTool
        st = ScreenshotTool()
        result = await st.capture()
        await self.broadcast(self.task_id, {
            "type": "screenshot",
            "data": result["screenshot_base64"]
        })
    except Exception:
        pass # Não quebra o agente por falha de screenshot

async def log(self, message: str, level: str = "info"):
    await self.broadcast(self.task_id, {"type": "log", "level": level, "messa
    await save_log(self.task_id, level, message)

async def set_status(self, status: AgentStatus):
    self.state.status = status
    await self.broadcast(self.task_id, {"type": "status", "status": status.va

async def request_confirmation(self, message: str) -> bool:
    self._confirmation_event.clear()
    await self.set_status(AgentStatus.WAITING_CONFIRMATION)
    await self.log(f"⏸ Aguardando confirmação: {message}", "warning")
    await self.broadcast(self.task_id, {"type": "confirm_request", "message":
    await self._confirmation_event.wait()
    return self._confirmation_approved

def confirm(self, approved: bool):
    self._confirmation_approved = approved
    self._confirmation_event.set()

def pause(self):
    self._paused = True

def resume(self):
    self._paused = False

```

```
def stop(self):
    self._stopped = True
    self._confirmation_event.set()
```

backend/agent/state.py

```
from enum import Enum
from dataclasses import dataclass

class AgentStatus(str, Enum):
    STOPPED = "stopped"
    THINKING = "thinking"
    EXECUTING = "executing"
    WAITING_CONFIRMATION = "waiting_confirmation"
    ERROR = "error"
    DONE = "done"

@dataclass
class AgentState:
    task_id: str
    status: AgentStatus = AgentStatus.STOPPED
```

7. Sistema de Ferramentas (Tools)

tools/browser.py — Playwright + Google Chrome via CDP

Esta é a parte mais importante adaptada para Linux Mint. Em vez de baixar o Chromium do Playwright, conectamos ao **Google Chrome já instalado** usando o Chrome DevTools Protocol (CDP).

```
import asyncio
import subprocess
import base64
from playwright.async_api import async_playwright, Browser, Page, BrowserContext
from config import settings

class BrowserTool:
    """
    Controla o Google Chrome instalado no sistema via CDP.
    Não baixa um navegador separado — usa o Chrome nativo do Linux Mint.
    """
    def __init__(self):
```

```

self._playwright = None
self._browser: Browser = None
self._context: BrowserContext = None
self._page: Page = None
self._chrome_process = None

async def _ensure_browser(self):
    """
    Estratégia dupla:
    1. Tenta conectar ao Chrome via CDP (se já estiver rodando com --remote-d
    2. Se não, lança o Chrome com a flag de debug e conecta
    """
    if self._page and not self._page.is_closed():
        return

    if not self._playwright:
        self._playwright = await async_playwright().start()

    try:
        # Tenta conectar ao Chrome existente na porta de debug
        self._browser = await self._playwright.chromium.connect_over_cdp(
            "http://localhost:9222"
        )
        contexts = self._browser.contexts
        if contexts:
            self._context = contexts[0]
            pages = self._context.pages
            self._page = pages[0] if pages else await self._context.new_page()
        else:
            self._context = await self._browser.new_context()
            self._page = await self._context.new_page()

    except Exception:
        # Chrome não está rodando com debug – lança um novo
        await self._launch_chrome()

async def _launch_chrome(self):
    """Lança o Google Chrome com a porta de debug aberta."""
    chrome_args = [
        settings.CHROME_BINARY,
        "--remote-debugging-port=9222",
        "--no-first-run",
        "--no-default-browser-check",
        f"--user-data-dir=/tmp/manus-chrome-profile",
    ]

    # Em Linux sem display físico (modo headless de servidor), adiciona:

```

```

# "--headless=new"
# Mas como você TEM display, deixa com GUI para ver o que o agente faz

self._chrome_process = subprocess.Popen(
    chrome_args,
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL
)

# Aguarda o Chrome inicializar
await asyncio.sleep(2)

self._browser = await self._playwright.chromium.connect_over_cdp(
    "http://localhost:9222"
)
self._context = await self._browser.new_context()
self._page = await self._context.new_page()

async def navigate(self, url: str, task_id: str = None) -> dict:
    await self._ensure_browser()
    try:
        await self._page.goto(url, wait_until="domcontentloaded", timeout=300)
        return {
            "url": self._page.url,
            "title": await self._page.title(),
            "status": "ok"
        }
    except Exception as e:
        return {"error": str(e)}

async def click(self, selector: str, task_id: str = None) -> dict:
    await self._ensure_browser()
    try:
        await self._page.click(selector, timeout=10000)
        await self._page.wait_for_load_state("domcontentloaded", timeout=5000)
        return {"clicked": selector, "status": "ok"}
    except Exception as e:
        return {"error": f"Não foi possível clicar em '{selector}': {e}"}

async def click_text(self, text: str, task_id: str = None) -> dict:
    """Clica em um elemento pelo texto visível – mais robusto que seletores C
    await self._ensure_browser()
    try:
        await self._page.get_by_text(text, exact=False).first.click(timeout=1)
        return {"clicked_text": text, "status": "ok"}
    except Exception as e:
        return {"error": f"Texto '{text}' não encontrado: {e}"}

```

```

async def type_text(self, selector: str, text: str, task_id: str = None) -> dict:
    await self._ensure_browser()
    try:
        await self._page.fill(selector, text)
        return {"typed": text, "in": selector}
    except Exception as e:
        return {"error": str(e)}

async def press_key(self, key: str, task_id: str = None) -> dict:
    """Ex: key='Enter', 'Tab', 'Escape'"""
    await self._ensure_browser()
    await self._page.keyboard.press(key)
    return {"pressed": key}

async def extract_text(self, task_id: str = None) -> dict:
    await self._ensure_browser()
    try:
        text = await self._page.inner_text("body")
        # Limita para não explodir o contexto da API
        return {"text": text[:6000], "url": self._page.url}
    except Exception as e:
        return {"error": str(e)}

async def extract_links(self, task_id: str = None) -> dict:
    await self._ensure_browser()
    links = await self._page.eval_on_selector_all(
        "a[href]",
        "els => els.map(e => ({ text: e.innerText.trim(), href: e.href })).sl
    )
    return {"links": links}

async def take_screenshot(self, task_id: str = None) -> dict:
    await self._ensure_browser()
    try:
        screenshot_bytes = await self._page.screenshot(type="jpeg", quality=75)
        b64 = base64.b64encode(screenshot_bytes).decode()
        return {"screenshot_base64": b64, "url": self._page.url}
    except Exception as e:
        return {"error": str(e)}

async def scroll(self, direction: str = "down", amount: int = 500, task_id: str = None) -> dict:
    await self._ensure_browser()
    y = amount if direction == "down" else -amount
    await self._page.mouse.wheel(0, y)
    await asyncio.sleep(0.5)
    return {"scrolled": direction, "amount": amount}

```



```

async def wait_for_element(self, selector: str, timeout: int = 10000, task_id
    await self._ensure_browser()
    try:
        await self._page.wait_for_selector(selector, timeout=timeout)
        return {"found": selector}
    except Exception as e:
        return {"error": f"Elemento não apareceu em {timeout}ms: {e}"}

async def evaluate_js(self, script: str, task_id: str = None) -> dict:
    """Executa JavaScript na página – útil para extrair dados estruturados."""
    await self._ensure_browser()
    try:
        result = await self._page.evaluate(script)
        return {"result": str(result)[:3000]}
    except Exception as e:
        return {"error": str(e)}

async def close(self):
    if self._browser:
        await self._browser.close()
    if self._chrome_process:
        self._chrome_process.terminate()

```

tools/shell.py — Terminal bash com Whitelist

No Linux Mint, o shell é bash, o que torna a whitelist mais poderosa e previsível.

```

import asyncio
import shlex
from pathlib import Path
from config import settings

# Comandos permitidos no bash do Linux Mint
# Mais rico que no Windows – apt, python3, curl funcionam nativamente
ALLOWED_COMMANDS = {
    # Python
    "python3", "python", "pip3", "pip",
    # Node/npm
    "node", "npm", "npmx",
    # Sistema de arquivos
    "ls", "cat", "head", "tail", "grep", "find", "wc",
    "mkdir", "cp", "mv", "rm", # rm só funciona na workspace
    "touch", "echo", "pwd",
    # Rede
    "curl", "wget",

```

```

# Desenvolvimento
"git", "pandoc", "ffmpeg",
# Conversão de documentos
"libreoffice", "convert", # ImageMagick
# Utilitários de texto
"sort", "uniq", "awk", "sed", "cut", "tr",
# Informações do sistema (somente leitura)
"df", "free", "uname",
}

# Comandos absolutamente proibidos (proteção extra)
BLOCKED_PATTERNS = [
    "sudo", "su ", "chmod", "chown", "passwd",
    "systemctl", "service", "kill", "killall",
    "dd ", "mkfs", "fdisk", "format",
    "rm -rf /", "rm -rf ~",
    "> /dev/", "| sh", "| bash",
    "curl.*|.*sh", # pipe de curl direto para shell
]

class ShellTool:
    async def run(self, command: str, timeout: int = 30, task_id: str = None) ->
        # Verifica padrões bloqueados (antes mesmo de parsear)
        for pattern in BLOCKED_PATTERNS:
            if pattern in command.lower():
                return {"error": f"Comando bloqueado por segurança: contém '{pattern}'"}

        parts = shlex.split(command)
        if not parts:
            return {"error": "Comando vazio"}

        base_cmd = Path(parts[0]).name.lower() # Pega só o nome, ignora caminho

        if base_cmd not in ALLOWED_COMMANDS:
            return {
                "error": f"Comando '{base_cmd}' não está na lista de permitidos."
                "dica": "Peça ao usuário para adicionar o comando à whitelist se
            }

        try:
            proc = await asyncio.create_subprocess_exec(
                *parts,
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE,
                cwd=str(settings.WORKSPACE_PATH) # sempre na workspace
            )

```

```

try:
    stdout, stderr = await asyncio.wait_for(
        proc.communicate(), timeout=timeout
    )
except asyncio.TimeoutError:
    proc.kill()
    return {"error": f"Timeout após {timeout}s"}

return {
    "stdout": stdout.decode("utf-8", errors="replace")[:4000],
    "stderr": stderr.decode("utf-8", errors="replace")[:500],
    "returncode": proc.returncode,
    "cwd": str(settings.WORKSPACE_PATH)
}

except FileNotFoundError:
    return {"error": f"Comando '{base_cmd}' não encontrado. Pode não esta
except Exception as e:
    return {"error": str(e)}

```

tools/file_manager.py — Gerenciador de Arquivos Isolado

```

from pathlib import Path
import aiofiles
import json
from config import settings

```

```

WORKSPACE = settings.WORKSPACE_PATH.resolve()

```

```

class FileManagerTool:

```

```

    def _safe_path(self, filename: str) -> Path:
        """Bloqueia path traversal – o agente não sai da workspace."""
        target = (WORKSPACE / filename).resolve()
        if not str(target).startswith(str(WORKSPACE)):
            raise PermissionError(f"Acesso negado: '{filename}' está fora da work
        return target

```

```

    async def read(self, filename: str, task_id: str = None) -> dict:

```

```

        try:
            path = self._safe_path(filename)
            if not path.exists():
                return {"error": f"Arquivo não encontrado: {filename}"}
            async with aiofiles.open(path, "r", encoding="utf-8", errors="replace") as f:
                content = await f.read()
            return {
                "content": content[:10000],

```

```

        "size_bytes": path.stat().st_size,
        "truncated": len(content) > 10000
    }
except PermissionError as e:
    return {"error": str(e)}

async def write(self, filename: str, content: str, task_id: str = None) -> dict:
    try:
        path = self._safe_path(filename)
        path.parent.mkdir(parents=True, exist_ok=True)
        async with aiofiles.open(path, "w", encoding="utf-8") as f:
            await f.write(content)
        return {
            "written": filename,
            "path": str(path),
            "bytes": len(content.encode("utf-8"))
        }
    except PermissionError as e:
        return {"error": str(e)}

async def append(self, filename: str, content: str, task_id: str = None) -> dict:
    try:
        path = self._safe_path(filename)
        path.parent.mkdir(parents=True, exist_ok=True)
        async with aiofiles.open(path, "a", encoding="utf-8") as f:
            await f.write(content)
        return {"appended_to": filename, "bytes_added": len(content.encode())}
    except PermissionError as e:
        return {"error": str(e)}

async def list_files(self, subdirectory: str = "", task_id: str = None) -> dict:
    base = self._safe_path(subdirectory) if subdirectory else WORKSPACE
    files = []
    for f in base.rglob("*"):
        if f.is_file():
            files.append({
                "name": str(f.relative_to(WORKSPACE)),
                "size_bytes": f.stat().st_size,
                "extension": f.suffix,
                "modified": f.stat().st_mtime
            })
    return {"files": files, "total": len(files), "workspace": str(WORKSPACE)}

async def delete(self, filename: str, task_id: str = None) -> dict:
    """Só deleta arquivos dentro da workspace."""
    try:
        path = self._safe_path(filename)

```

```

    if path.is_file():
        path.unlink()
        return {"deleted": filename}
    return {"error": f"Arquivo não encontrado: {filename}"}
except PermissionError as e:
    return {"error": str(e)}

```

tools/screenshot.py — Captura de Tela via X11

No Linux Mint com X11, o MSS funciona nativamente sem configuração extra.

```

import mss
import base64
from io import BytesIO
from PIL import Image
import os

class ScreenshotTool:
    async def capture(self, monitor: int = 1, task_id: str = None) -> dict:
        """
        Captura a tela via X11 (Linux Mint).
        Funciona automaticamente com o servidor X do Mint.
        """
        # Garante que DISPLAY está definido (necessário quando rodando via system
        if "DISPLAY" not in os.environ:
            os.environ["DISPLAY"] = ":0"

        try:
            with mss.mss() as sct:
                # monitor=0 = todos os monitores, monitor=1 = monitor principal
                mon = sct.monitors[monitor] if monitor < len(sct.monitors) else s
                screenshot = sct.grab(mon)

                img = Image.frombytes(
                    "RGB",
                    screenshot.size,
                    screenshot.bgra,
                    "raw", "BGRX"
                )

                # Redimensiona para economizar tokens nas APIs de visão
                # 1280x720 é suficiente para a IA entender o conteúdo
                img.thumbnail((1280, 720), Image.LANCZOS)

                buffer = BytesIO()
                img.save(buffer, format="JPEG", quality=80)

```

```

        b64 = base64.b64encode(buffer.getvalue()).decode()

    return {
        "screenshot_base64": b64,
        "width": img.width,
        "height": img.height,
        "monitor": monitor
    }

except Exception as e:
    return {"error": f"Falha ao capturar tela: {e}. Verifique se DISPLAY="

async def capture_region(self, x: int, y: int, width: int, height: int,
                        task_id: str = None) -> dict:
    """Captura uma região específica da tela."""
    if "DISPLAY" not in os.environ:
        os.environ["DISPLAY"] = ":0"

    with mss.mss() as sct:
        region = {"top": y, "left": x, "width": width, "height": height}
        screenshot = sct.grab(region)
        img = Image.frombytes("RGB", screenshot.size, screenshot.bgra, "raw",
                             buffer = BytesIO())
        img.save(buffer, format="JPEG", quality=85)
        b64 = base64.b64encode(buffer.getvalue()).decode()

    return {"screenshot_base64": b64, "region": region}

```

tools/pyautogui_tool.py — Mouse e Teclado fora do Navegador

No Linux Mint, PyAutoGUI usa a biblioteca Xlib, que funciona nativamente.

```

import pyautogui
import asyncio
import os

# Configurações de segurança do PyAutoGUI
pyautogui.FAILSAFE = True    # Mover mouse para canto superior esquerdo = para tud
pyautogui.PAUSE = 0.3       # Pausa entre ações (evita erros por velocidade)

class PyAutoGUITool:
    def __init__(self):
        if "DISPLAY" not in os.environ:
            os.environ["DISPLAY"] = ":0"

    async def move_mouse(self, x: int, y: int, duration: float = 0.5,

```

```

        task_id: str = None) -> dict:
    await asyncio.get_event_loop().run_in_executor(
        None, lambda: pyautogui.moveTo(x, y, duration=duration)
    )
    return {"moved_to": {"x": x, "y": y}}

async def click_at(self, x: int, y: int, button: str = "left",
        task_id: str = None) -> dict:
    await asyncio.get_event_loop().run_in_executor(
        None, lambda: pyautogui.click(x, y, button=button)
    )
    return {"clicked_at": {"x": x, "y": y, "button": button}}

async def type_string(self, text: str, interval: float = 0.05,
        task_id: str = None) -> dict:
    await asyncio.get_event_loop().run_in_executor(
        None, lambda: pyautogui.typewrite(text, interval=interval)
    )
    return {"typed": text}

async def hotkey(self, *keys: str, task_id: str = None) -> dict:
    """Ex: hotkey('ctrl', 'c') para copiar."""
    await asyncio.get_event_loop().run_in_executor(
        None, lambda: pyautogui.hotkey(*keys)
    )
    return {"hotkey": list(keys)}

async def screenshot_position(self, task_id: str = None) -> dict:
    """Retorna a posição atual do mouse."""
    x, y = pyautogui.position()
    return {"mouse_x": x, "mouse_y": y}

```

8. Integração com IA — DeepSeek + Visão

agent/planner.py — DeepSeek com Tool Calling

```

import httpx
import json
from config import settings

```

```

SYSTEM_PROMPT = """Você é um agente de automação rodando em Linux Mint com Google
Sempre responda com um JSON válido no seguinte formato:

```

```
{
  "action": "nome_da_ferramenta",
  "description": "O que você está fazendo e por quê (explique seu raciocínio)",
  "params": { ... },
  "requires_confirmation": false,
  "confirmation_message": ""
}
```

Ou, quando a tarefa estiver concluída:

```
{
  "action": "finish",
  "result": "Resumo do que foi feito e onde os arquivos foram salvos"
}
```

Ferramentas disponíveis

Navegador (Google Chrome via CDP)

Ferramenta	Parâmetros	Descrição
browser_navigate	url: str	Navega para URL
browser_click	selector: str	Clica por seletor CSS
browser_click_text	text: str	Clica por texto visível (preferível ao CSS)
browser_type	selector: str, text: str	Digita em campo
browser_press_key	key: str	Pressiona tecla (Enter, Tab, Escape, etc)
browser_extract_text	(nenhum)	Extrai todo o texto da página
browser_extract_links	(nenhum)	Lista links da página
browser_screenshot	(nenhum)	Tira print do Chrome
browser_scroll	direction: "up"/"down", amount: int	Rola a página
browser_wait_for_element	selector: str, timeout: int	Aguarda elemento aparecer
browser_evaluate_js	script: str	Executa JavaScript na página

Terminal bash (whitelist)

Ferramenta	Parâmetros	Descrição
shell_run	command: str, timeout: int	Executa comando permitido

Comandos permitidos: python3, pip3, node, npm, git, curl, wget, ls, cat, grep, fi

Arquivos (workspace isolada em ~/manus-workspace/)

Ferramenta	Parâmetros	Descrição
file_read	filename: str	Lê arquivo
file_write	filename: str, content: str	Cria/sobrescreve arquivo
file_append	filename: str, content: str	Adiciona ao final
file_list	subdirectory: str (opcional)	Lista arquivos
file_delete	filename: str	Deleta arquivo


```

### Captura de tela (X11)
| Ferramenta | Parâmetros | Descrição |
|---|---|---|
| screenshot_capture | monitor: int (padrão 1) | Captura tela inteira |
| screenshot_region | x,y,width,height: int | Captura região específica |

```

```

### Visão por IA
| Ferramenta | Parâmetros | Descrição |
|---|---|---|
| vision_analyze | image_base64: str, question: str | Analisa imagem |

```

```

### Mouse e teclado (fora do navegador)
| Ferramenta | Parâmetros | Descrição |
|---|---|---|
| pyautogui_click | x: int, y: int | Clica em coordenada |
| pyautogui_type | text: str | Digita texto |
| pyautogui_hotkey | keys: list[str] | Executa atalho de teclado |

```

Regras obrigatórias

1. Sempre que não souber o estado visual atual, use vision_analyze com o resultado
2. Prefira browser_click_text ao browser_click com CSS — é mais robusto
3. Ações destrutivas (delete, enviar formulário com dados reais) DEVEM ter requir
4. Salve arquivos importantes na workspace antes de declarar "finish"
5. Um JSON por resposta, sem texto fora do JSON
6. Se um passo falhar, reavalie e tente uma abordagem diferente""

```

class DeepSeekPlanner:
    async def get_next_action(self, conversation_history: list) -> dict:
        messages = [
            {"role": "system", "content": SYSTEM_PROMPT},
            *conversation_history
        ]

        async with httpx.AsyncClient(timeout=90.0) as client:
            response = await client.post(
                "https://api.deepseek.com/chat/completions",
                headers={
                    "Authorization": f"Bearer {settings.DEEPSEEK_API_KEY}",
                    "Content-Type": "application/json"
                },
                json={
                    "model": "deepseek-chat",
                    "messages": messages,
                    "temperature": 0.1,
                    "max_tokens": 1500,
                    "response_format": {"type": "json_object"}
                }
            )

```

```

    )
    response.raise_for_status()
    data = response.json()
    content = data["choices"][0]["message"]["content"]
    return json.loads(content)

```

tools/vision.py — Análise de Imagem Plugável

```

import httpx
from config import settings

class VisionTool:
    """
    Troque o provider no .env sem mudar nenhum outro arquivo:
    VISION_PROVIDER=openai    → usa GPT-4o Mini
    VISION_PROVIDER=google    → usa Gemini 1.5 Flash
    """

    async def analyze(self, image_base64: str, question: str, task_id: str = None
        provider = settings.VISION_PROVIDER
        if provider == "openai":
            return await self._openai(image_base64, question)
        elif provider == "google":
            return await self._google(image_base64, question)
        else:
            raise ValueError(f"VISION_PROVIDER inválido: '{provider}'. Use 'opena

    async def _openai(self, image_base64: str, question: str) -> dict:
        async with httpx.AsyncClient(timeout=30.0) as client:
            r = await client.post(
                "https://api.openai.com/v1/chat/completions",
                headers={"Authorization": f"Bearer {settings.OPENAI_API_KEY}"},
                json={
                    "model": "gpt-4o-mini",
                    "messages": [{
                        "role": "user",
                        "content": [
                            {"type": "image_url", "image_url": {
                                "url": f"data:image/jpeg;base64,{image_base64}",
                                "detail": "low"    # "low" = mais barato, suficien
                            }},
                            {"type": "text", "text": question}
                        ]
                    }],
                    "max_tokens": 600
                }
            )

```

```

    )
    r.raise_for_status()
    data = r.json()
    return {"analysis": data["choices"][0]["message"]["content"], "provider": "openai"}

async def _google(self, image_base64: str, question: str) -> dict:
    async with httpx.AsyncClient(timeout=30.0) as client:
        r = await client.post(
            f"https://generativelanguage.googleapis.com/v1beta/models/gemini-1.0-pro-latest:generateContent?key={self.api_key}&alt=json={
                "contents": [{
                    "parts": [
                        {"inline_data": {"mime_type": "image/jpeg", "data": image_base64}},
                        {"text": question}
                    ]
                }],
                "generationConfig": {"maxOutputTokens": 600}
            }
        )
        r.raise_for_status()
        data = r.json()
        return {
            "analysis": data["candidates"][0]["content"]["parts"][0]["text"],
            "provider": "google"
        }

```

9. Frontend — Interface Web

A interface usa **React + Vite + Tailwind CSS**, buildada uma vez e servida como arquivos estáticos pelo FastAPI. Nenhum servidor Node roda em produção.

```

// frontend/src/App.jsx
import { useState, useEffect, useRef } from "react";
import { useWebSocket } from "../hooks/useWebSocket";

const STATUS_CONFIG = {
    stopped: { label: "Parado", color: "bg-gray-600", },
    thinking: { label: "Pensando...", color: "bg-blue-600 animate-pulse", },
    executing: { label: "Executando", color: "bg-yellow-600", },
    waiting_confirmation: { label: "Aguardando você", color: "bg-orange-600 animate-pulse", },
    error: { label: "Erro", color: "bg-red-600", },
    done: { label: "Concluído", color: "bg-green-600", },
};

```

```

const LOG_COLORS = {
  info: "text-gray-300",
  action: "text-blue-400",
  result: "text-cyan-400",
  success: "text-green-400",
  warning: "text-yellow-400",
  error: "text-red-400",
};

export default function App() {
  const [task, setTask] = useState("");
  const [currentTaskId, setCurrentTaskId] = useState(null);
  const [status, setStatus] = useState("stopped");
  const [logs, setLogs] = useState([]);
  const [screenshot, setScreenshot] = useState(null);
  const [files, setFiles] = useState([]);
  const [confirmRequest, setConfirmRequest] = useState(null);
  const [token, setToken] = useState(localStorage.getItem("auth_token"));
  const [loginForm, setLoginForm] = useState({ user: "", pass: "" });
  const logsEndRef = useRef(null);

  const { connected } = useWebSocket(currentTaskId, token, (msg) => {
    if (msg.type === "log") {
      setLogs(prev => [...prev.slice(-500), { ...msg, time: new Date().toLocaleTi
    }
    if (msg.type === "status") setStatus(msg.status);
    if (msg.type === "screenshot") setScreenshot(msg.data);
    if (msg.type === "confirm_request") setConfirmRequest(msg.message);
    if (msg.type === "files_update") setFiles(msg.files);
  });

  useEffect(() => {
    logsEndRef.current?.scrollIntoView({ behavior: "smooth" });
  }, [logs]);

  const login = async () => {
    const res = await fetch(`/auth/login?username=${loginForm.user}&password=${lo
      method: "POST"
    });
    if (res.ok) {
      const data = await res.json();
      localStorage.setItem("auth_token", data.token);
      setToken(data.token);
    } else {
      alert("Credenciais inválidas");
    }
  };
};

```

```

const startTask = async () => {
  if (!task.trim()) return;
  const res = await fetch("/api/tasks/", {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${tok
    body: JSON.stringify({ description: task })
  });
  const data = await res.json();
  setCurrentTaskId(data.task_id);
  setLogs([]);
  setFiles([]);
  setScreenshot(null);
  setStatus("thinking");
};

const sendControl = async (action) => {
  if (!currentTaskId) return;
  await fetch(`/api/control/${currentTaskId}/${action}`, {
    method: "POST",
    headers: { Authorization: `Bearer ${token}` }
  });
};

const handleConfirm = async (approved) => {
  await fetch(`/api/control/${currentTaskId}/confirm?approved=${approved}`, {
    method: "POST",
    headers: { Authorization: `Bearer ${token}` }
  });
  setConfirmRequest(null);
};

// Tela de Login
if (!token) {
  return (
    <div className="min-h-screen bg-gray-950 flex items-center justify-center">
      <div className="bg-gray-800 p-8 rounded-2xl w-full max-w-sm border border
        <h1 className="text-2xl font-bold text-center mb-6">👤 Manus Local</h1>
        <input
          type="text" placeholder="Usuário"
          value={loginForm.user}
          onChange={e => setLoginForm(p => ({...p, user: e.target.value}))}
          className="w-full bg-gray-700 rounded-lg p-3 mb-3 focus:outline-none
        />
        <input
          type="password" placeholder="Senha"
          value={loginForm.pass}

```

```

        onChange={e => setLoginForm(p => ({...p, pass: e.target.value}))}
        onKeyDown={e => e.key === "Enter" && login()}
        className="w-full bg-gray-700 rounded-lg p-3 mb-4 focus:outline-none
    />
    <button onClick={login}
        className="w-full py-3 bg-blue-600 hover:bg-blue-500 rounded-lg font-
        Entrar
    </button>
</div>
</div>
);
}

```

```
const sc = STATUS_CONFIG[status] || STATUS_CONFIG.stopped;
```

```

return (
    <div className="min-h-screen bg-gray-950 text-white flex flex-col font-sans">
        {/* Header */}
        <header className="bg-gray-900 px-6 py-3 flex items-center justify-between">
            <div className="flex items-center gap-3">
                <span className="text-xl font-bold">🧑‍🎓 Manus Local</span>
                <span className="text-xs text-gray-500">Linux Mint + Chrome</span>
            </div>
            <div className="flex items-center gap-3">
                <span className={`px-3 py-1 rounded-full text-xs font-semibold ${sc.col}
                    {sc.icon} {sc.label}`}>
                </span>
                <button onClick={() => sendControl("pause")}
                    disabled={status === "stopped" || status === "done"}
                    className="px-3 py-1 bg-yellow-700 hover:bg-yellow-600 disabled:opaci
                        🛑 Pausar
                </button>
                <button onClick={() => sendControl("resume")}
                    disabled={status !== "thinking" && status !== "executing"}
                    className="px-3 py-1 bg-blue-700 hover:bg-blue-600 disabled:opacity-4
                        ▶ Retomar
                </button>
                <button onClick={() => sendControl("stop")}
                    disabled={status === "stopped" || status === "done"}
                    className="px-3 py-1 bg-red-700 hover:bg-red-600 disabled:opacity-40
                        🛑 Parar
                </button>
                <button onClick={() => { localStorage.removeItem("auth_token"); setToke
                    className="px-3 py-1 bg-gray-700 hover:bg-gray-600 rounded-lg text-sm
                    Sair
                </button>
            </div>
        </div>
    </div>

```

```

</header>

{/* Modal de Confirmação */}
{confirmRequest && (
  <div className="fixed inset-0 bg-black/80 flex items-center justify-cente
    <div className="bg-gray-800 p-6 rounded-2xl max-w-lg w-full border-2 bo
      <h2 className="text-lg font-bold text-orange-400 mb-2">⚠ O agente pr
      <p className="text-gray-200 mb-6 leading-relaxed">{confirmRequest}</p
      <div className="flex gap-3">
        <button onClick={() => handleConfirm(true)}
          className="flex-1 py-3 bg-green-600 hover:bg-green-500 rounded-xl
             Confirmar e continuar
        </button>
        <button onClick={() => handleConfirm(false)}
          className="flex-1 py-3 bg-red-700 hover:bg-red-600 rounded-xl fon
             Cancelar ação
        </button>
      </div>
    </div>
  </div>
)}

{/* Layout principal */}
<div className="flex flex-1 overflow-hidden">
  {/* Coluna esquerda */}
  <div className="w-80 shrink-0 flex flex-col gap-3 p-4 border-r border-gra
    {/* Input de tarefa */}
    <div className="bg-gray-900 rounded-xl p-4 border border-gray-800">
      <h2 className="text-xs font-semibold text-gray-500 uppercase tracking
        Nova Tarefa
      </h2>
      <textarea
        value={task}
        onChange={e => setTask(e.target.value)}
        placeholder="Descreva o que o agente deve fazer em linguagem natura
        className="w-full bg-gray-800 rounded-lg p-3 text-sm resize-none h-
      />
      <button
        onClick={startTask}
        disabled={!task.trim() || status === "executing" || status === "thi
        className="w-full mt-2 py-2.5 bg-blue-600 hover:bg-blue-500 disable
           Iniciar Tarefa
      </button>
    </div>

  {/* Arquivos gerados */}
  <div className="bg-gray-900 rounded-xl p-4 border border-gray-800 flex-

```

```

    <h2 className="text-xs font-semibold text-gray-500 uppercase tracking
      Arquivos Gerados
    </h2>
    {files.length === 0 ? (
      <p className="text-gray-600 text-xs">Nenhum arquivo ainda</p>
    ) : (
      <ul className="space-y-1.5">
        {files.map(f => (
          <li key={f.name}
            className="flex items-center justify-between bg-gray-800 roun
            <span className="truncate text-gray-300">{f.name}</span>
            <a
              href={`\api/files/${encodeURIComponent(f.name)}\`}
              download
              className="text-blue-400 hover:text-blue-300 shrink-0 font-
                📄 baixar
            </a>
          </li>
        ))}
      </ul>
    )}
  </div>
</div>

{/* Coluna central - screenshot */}
<div className="flex-1 flex items-center justify-center p-4 bg-gray-950 o
  {screenshot ? (
    <img
      src={`data:image/jpeg;base64,${screenshot}`}
      alt="Estado atual da tela"
      className="max-w-full max-h-full rounded-xl shadow-lg object-contai
    />
  ) : (
    <div className="text-center text-gray-700">
      <div className="text-6xl mb-4">🖥️</div>
      <p className="text-sm">O estado da tela aparecerá aqui durante a ex
      <p className="text-xs mt-2 text-gray-800">Screenshots são capturado
    </div>
  )}
</div>
</div>

{/* Painel de logs */}
<div className="h-44 bg-black border-t border-gray-800 flex flex-col shrink
  <div className="px-4 py-1.5 border-b border-gray-900 flex items-center ju
    <span className="text-xs font-semibold text-gray-600 uppercase tracking
      Logs em Tempo Real

```



```

        </span>
        <button onClick={() => setLogs([])}
            className="text-xs text-gray-700 hover:text-gray-500">limpar</button>
    </div>
    <div className="flex-1 overflow-y-auto p-3 font-mono text-xs space-y-0.5"
        {logs.map((log, i) => (
            <div key={i} className={LOG_COLORS[log.level] || "text-gray-400"}>
                <span className="text-gray-700 mr-2">[{log.time}]</span>
                {log.message}
            </div>
        ))}
        <div ref={logsEndRef} />
    </div>
</div>
</div>
);
}

```

10. WebSocket — Logs em Tempo Real

backend/api/ws.py

```

from fastapi import APIRouter, WebSocket, WebSocketDisconnect, Query
from typing import Dict, List
import json
import jwt
from config import settings

router = APIRouter()
active_connections: Dict[str, List[WebSocket]] = {}

async def broadcast_to_task(task_id: str, message: dict):
    if task_id not in active_connections:
        return
    dead = []
    for ws in active_connections[task_id]:
        try:
            await ws.send_json(message)
        except Exception:
            dead.append(ws)
    for ws in dead:
        active_connections[task_id].remove(ws)

```

```

@router.websocket("/ws/{task_id}")
async def websocket_endpoint(
    websocket: WebSocket,
    task_id: str,
    token: str = Query(...) # token passado como query param: /ws/123?token=...
):
    # Autenticação do WebSocket
    try:
        jwt.decode(token, settings.SECRET_KEY, algorithms=["HS256"])
    except Exception:
        await websocket.close(code=4001, reason="Token inválido")
        return

    await websocket.accept()
    active_connections.setdefault(task_id, []).append(websocket)

    try:
        while True:
            await websocket.receive_text() # Mantém conexão viva
    except WebSocketDisconnect:
        if task_id in active_connections:
            try:
                active_connections[task_id].remove(websocket)
            except ValueError:
                pass

```

frontend/src/hooks/useWebSocket.js

```

import { useEffect, useRef, useState } from "react";

export function useWebSocket(taskId, token, onMessage) {
    const ws = useRef(null);
    const [connected, setConnected] = useState(false);
    const reconnectTimer = useRef(null);

    useEffect(() => {
        if (!taskId || !token) return;

        const connect = () => {
            const protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
            const url = `${protocol}://${window.location.host}/ws/${taskId}?token=${token}`;

            ws.current = new WebSocket(url);

            ws.current.onopen = () => setConnected(true);
            ws.current.onclose = () => {
                setConnected(false);
            }
        }
    }, [taskId, token]);

```

```

        // Reconecta após 3 segundos se a tarefa ainda estiver ativa
        reconnectTimer.current = setTimeout(connect, 3000);
    };
    ws.current.onerror = () => ws.current.close();
    ws.current.onmessage = (event) => {
        try { onMessage(JSON.parse(event.data)); }
        catch (e) { console.error("WS parse error:", e); }
    };
};

connect();

return () => {
    clearTimeout(reconnectTimer.current);
    ws.current?.close();
};
}, [taskId, token]));

return { connected };
}

```

11. Segurança e Autenticação

backend/auth.py

```

from fastapi import APIRouter, HTTPException, Depends
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt
from datetime import datetime, timedelta, timezone
from config import settings

auth_router = APIRouter()
security = HTTPBearer()

def create_token(username: str) -> str:
    payload = {
        "sub": username,
        "exp": datetime.now(timezone.utc) + timedelta(hours=24),
        "iat": datetime.now(timezone.utc)
    }
    return jwt.encode(payload, settings.SECRET_KEY, algorithm="HS256")

def verify_token(credentials: HTTPAuthorizationCredentials = Depends(security)) -

```

```

try:
    payload = jwt.decode(
        credentials.credentials,
        settings.SECRET_KEY,
        algorithms=["HS256"]
    )
    return payload["sub"]
except jwt.ExpiredSignatureError:
    raise HTTPException(status_code=401, detail="Token expirado – faça login")
except jwt.InvalidTokenError:
    raise HTTPException(status_code=401, detail="Token inválido")

@auth_router.post("/login")
async def login(username: str, password: str):
    if username != settings.ADMIN_USER or password != settings.ADMIN_PASS:
        raise HTTPException(status_code=401, detail="Usuário ou senha incorretos")
    return {
        "token": create_token(username),
        "token_type": "bearer",
        "expires_in": "24h"
    }

```

Checklist de Segurança — Linux Mint

```

# Gere a SECRET_KEY
openssl rand -hex 32

# Restrinja permissões do .env (só você pode ler)
chmod 600 ~/manus-local/.env

# Restrinja permissões da workspace
chmod 750 ~/manus-workspace/

# Verifique que o .env está no .gitignore
echo ".env" >> ~/manus-local/.gitignore

```

12. Cloudflare Tunnel — Exposição Externa

Instalação no Linux Mint (Debian/Ubuntu base)

```

# Método 1: via .deb (recomendado para Linux Mint)
curl -L https://pkg.cloudflare.com/cloudflare-main.gpg | sudo gpg --dearmor -o /u

```

```
echo "deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare
sudo tee /etc/apt/sources.list.d/cloudflared.list

sudo apt update
sudo apt install cloudflared

# Verifica instalação
cloudflared --version
```

Configuração Completa

```
# 1. Autentique com sua conta Cloudflare
cloudflared login
# Isso abre o Chrome e pede para você autorizar – escolha seu domínio

# 2. Crie o tunnel
cloudflared tunnel create manus-agent
# Anote o UUID gerado (ex: alb2c3d4-...)

# 3. Configure o DNS
cloudflared tunnel route dns manus-agent agente.seudominio.com
```

cloudflared-config.yml

```
tunnel: alb2c3d4-e5f6-7890-abcd-ef1234567890 # seu UUID
credentials-file: /home/SEU_USUARIO/.cloudflared/alb2c3d4-e5f6-7890-abcd-ef123456

ingress:
  - hostname: agente.seudominio.com
    service: http://127.0.0.1:8000
    originRequest:
      connectTimeout: 30s
      noTLSVerify: false
  - service: http_status:404
```

Teste antes de virar serviço

```
# Teste manual primeiro
cloudflared tunnel --config ~/manus-local/cloudflared-config.yml run

# Acesse https://agente.seudominio.com e veja se funciona
# Ctrl+C para parar após o teste
```

13. Banco de Dados — SQLite no HD de Backup

Conceito e Estrutura

Todo banco de dados do agente é salvo no **HD de backup conectado ao PC**, nunca no disco principal. O sistema:

1. **Detecta automaticamente** o HD de backup ao iniciar (varre `/media/$USER/` e `/mnt/`)
2. **Inspeciona** os bancos SQLite já existentes lá para seguir o mesmo padrão de tabelas
3. **Cria a pasta** `vortex/` dentro da pasta de bancos de dados do HD
4. **Cria uma subpasta por sessão** (`ses_1/` , `ses_2/` , `ses_3/` ...) a cada vez que o agente é iniciado
5. Cada sessão tem seu próprio arquivo `.db` isolado

Estrutura de Pastas no HD de Backup

```
/media/SEU_USUARIO/HD_BACKUP/  
└─ databases/                               ← pasta de bancos que já existe no HD  
    │└─ outro_banco.db                       ← bancos pré-existent (respeitados, não tocados)  
    │└─ outro_banco2.db  
    └─ Vortex/                               ← criada pelo agente na primeira execução  
        │└─ ses_1/                           ← primeira vez que o agente rodou  
            │└─ session.db  
        │└─ ses_2/                           ← segunda vez  
            │└─ session.db  
        │└─ ses_3/  
            │└─ session.db  
        └─ ...                               ← cresce infinitamente, uma pasta por execução
```

Como o agente descobre a pasta de bancos? Ele varre os pontos de montagem de HDs externos (`/media/$USER/*` e `/mnt/*`), encontra aquele que contém uma pasta com arquivos `.db` ou `.sqlite` , e usa essa pasta como base. Se encontrar mais de um candidato, usa o primeiro encontrado e registra no log qual foi escolhido.

```

import aiosqlite
import os
import re
from pathlib import Path
from typing import Optional

# -----
# DETECÇÃO DO HD DE BACKUP
# -----

def _find_backup_db_folder() -> Optional[Path]:
    """
    Varre os pontos de montagem comuns no Linux Mint em busca do HD de backup.
    Procura por uma pasta que contenha arquivos .db ou .sqlite (padrão pré-existe)
    Retorna o Path da pasta de bancos de dados encontrada, ou None se não achar.
    """
    user = os.environ.get("USER") or os.environ.get("LOGNAME") or "user"

    # Raízes onde HDs externos são montados no Linux Mint
    search_roots = [
        Path(f"/media/{user}"),
        Path("/media"),
        Path("/mnt"),
    ]

    candidates = []

    for root in search_roots:
        if not root.exists():
            continue
        # Percorre até 3 níveis de profundidade dentro do ponto de montagem
        for item in root.rglob("*"):
            if not item.is_dir():
                continue
            # Verifica se esta pasta contém arquivos de banco de dados
            db_files = list(item.glob("*.db")) + list(item.glob("*.sqlite"))
            if db_files:
                candidates.append((item, len(db_files)))

    if not candidates:
        return None

    # Prefere a pasta com mais arquivos .db (mais provável ser a pasta principal)
    candidates.sort(key=lambda x: x[1], reverse=True)
    return candidates[0][0]

```

```

def _detect_existing_schema(db_folder: Path) -> dict:
    """
    Inspecciona os bancos SQLite pré-existent no HD de backup.
    Retorna um dict com o padrão detectado (extensão, estrutura de tabelas, etc).
    Isso garante que o Vortex segue o mesmo padrão visual dos bancos já existente
    """
    import sqlite3

    schema_info = {
        "extension": ".db",    # padrão detectado
        "existing_tables": [],
    }

    db_files = list(db_folder.glob("*.db")) + list(db_folder.glob("*.sqlite"))

    # Prefere .sqlite se a maioria dos bancos usar essa extensão
    sqlite_count = len(list(db_folder.glob("*.sqlite")))
    db_count      = len(list(db_folder.glob("*.db")))
    if sqlite_count > db_count:
        schema_info["extension"] = ".sqlite"

    # Inspecciona o primeiro banco encontrado para entender as tabelas
    for db_file in db_files[:1]:
        try:
            conn = sqlite3.connect(str(db_file))
            cursor = conn.execute("SELECT name FROM sqlite_master WHERE type='tab'")
            schema_info["existing_tables"] = [row[0] for row in cursor.fetchall()]
            conn.close()
        except Exception:
            pass

    return schema_info

# -----
# GERENCIADOR DE SESSÕES – Vortex/ses_N/
# -----

class SessionManager:
    """
    Gerencia a estrutura de pastas no HD de backup:
    <pasta_de_bancos>/Vortex/ses_1/session.db
    <pasta_de_bancos>/Vortex/ses_2/session.db
    ...
    Cada execução do agente cria uma nova pasta ses_N.
    """

```



```

_instance: Optional["SessionManager"] = None

def __init__(self):
    self.backup_db_folder: Optional[Path] = None
    self.vortex_root: Optional[Path] = None
    self.session_folder: Optional[Path] = None
    self.session_db_path: Optional[Path] = None
    self.session_number: int = 0
    self.schema_info: dict = {}
    self._fallback_path: Path = Path.home() / "manus-local" / "fa
    self._initialized: bool = False

@classmethod
def get(cls) -> "SessionManager":
    if cls._instance is None:
        cls._instance = cls()
    return cls._instance

def initialize(self) -> dict:
    """
    Chamado uma vez na startup do FastAPI.
    Detecta o HD, cria Vortex/ e a pasta ses_N desta execução.
    Retorna um dict com o resultado para logging.
    """
    if self._initialized:
        return {"already_initialized": True}

    result = {}

    # 1. Detecta a pasta de bancos no HD de backup
    self.backup_db_folder = _find_backup_db_folder()

    if self.backup_db_folder is None:
        # HD de backup não encontrado – usa fallback local com aviso
        self.backup_db_folder = self._fallback_path
        self.backup_db_folder.mkdir(parents=True, exist_ok=True)
        result["warning"] = (
            "HD de backup não encontrado. "
            f"Usando pasta local de fallback: {self.backup_db_folder}. "
            "Conecte o HD de backup e reinicie o agente."
        )
    else:
        result["backup_folder_found"] = str(self.backup_db_folder)

    # 2. Inspecciona o padrão existente
    self.schema_info = _detect_existing_schema(self.backup_db_folder)

```

```

result["detected_extension"] = self.schema_info["extension"]
result["existing_tables_sample"] = self.schema_info["existing_tables"]

# 3. Cria (ou encontra) a pasta Vortex/
self.vortex_root = self.backup_db_folder / "Vortex"
self.vortex_root.mkdir(parents=True, exist_ok=True)
result["vortex_root"] = str(self.vortex_root)

# 4. Determina o próximo número de sessão
#     Conta pastas existentes: ses_1, ses_2, ... ses_N
existing_sessions = [
    d for d in self.vortex_root.iterdir()
    if d.is_dir() and re.match(r"^ses_\d+$", d.name)
]
if existing_sessions:
    max_n = max(int(d.name.split("_")[1]) for d in existing_sessions)
    self.session_number = max_n + 1
else:
    self.session_number = 1

# 5. Cria a pasta desta sessão
self.session_folder = self.vortex_root / f"ses_{self.session_number}"
self.session_folder.mkdir(parents=True, exist_ok=True)

# 6. Define o caminho do banco desta sessão
#     Segue a extensão detectada nos bancos pré-existent
self.session_db_path = self.session_folder / f"session{self.schema_info['

result["session_number"] = self.session_number
result["session_folder"] = str(self.session_folder)
result["session_db_path"] = str(self.session_db_path)

self._initialized = True
return result

@property
def db_path(self) -> Path:
    """Caminho do .db desta sessão. Usado por todas as funções do banco."""
    if not self._initialized:
        raise RuntimeError("SessionManager não foi inicializado. Chame .init")
    return self.session_db_path

def session_info(self) -> dict:
    return {
        "session_number": self.session_number,
        "session_folder": str(self.session_folder),
        "db_path": str(self.session_db_path),
    }

```

```

        "vortex_root": str(self.vortex_root),
        "backup_db_folder": str(self.backup_db_folder),
    }

# -----
# FUNÇÕES DO BANCO DE DADOS
# -----

async def init_db() -> dict:
    """
    Inicializa o SessionManager, cria a pasta ses_N no HD de backup,
    e cria as tabelas no banco desta sessão.
    Retorna informações de diagnóstico para o log de startup.
    """
    sm = SessionManager.get()
    init_result = sm.initialize()

    db_path = sm.db_path

    async with aiosqlite.connect(db_path) as db:
        await db.executescript("""
            -- Metadados desta sessão
            CREATE TABLE IF NOT EXISTS session_meta (
                key    TEXT PRIMARY KEY,
                value TEXT
            );

            -- Tarefas executadas nesta sessão
            CREATE TABLE IF NOT EXISTS tasks (
                id            TEXT PRIMARY KEY,
                description TEXT NOT NULL,
                status        TEXT DEFAULT 'pending',
                created_at    TEXT DEFAULT (datetime('now', 'localtime')),
                finished_at TEXT
            );

            -- Logs de cada tarefa
            CREATE TABLE IF NOT EXISTS logs (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                task_id       TEXT NOT NULL,
                level         TEXT NOT NULL,
                message       TEXT NOT NULL,
                created_at    TEXT DEFAULT (datetime('now', 'localtime')),
                FOREIGN KEY (task_id) REFERENCES tasks(id)
            );
        """)

```

```

-- Arquivos gerados pelo agente nesta sessão
CREATE TABLE IF NOT EXISTS generated_files (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    task_id       TEXT NOT NULL,
    filename      TEXT NOT NULL,
    size_bytes    INTEGER,
    created_at    TEXT DEFAULT (datetime('now', 'localtime')),
    FOREIGN KEY (task_id) REFERENCES tasks(id)
);

CREATE INDEX IF NOT EXISTS idx_logs_task_id ON logs(task_id);
CREATE INDEX IF NOT EXISTS idx_files_task_id ON generated_files(task_id)
)"""

# Salva metadados da sessão dentro do próprio banco
import json
from datetime import datetime
meta = {
    "session_number": str(sm.session_number),
    "started_at":     datetime.now().isoformat(),
    "vortex_root":    str(sm.vortex_root),
    "backup_folder":  str(sm.backup_db_folder),
}
for key, value in meta.items():
    await db.execute(
        "INSERT OR REPLACE INTO session_meta (key, value) VALUES (?, ?)",
        (key, value)
    )

await db.commit()

return init_result # Retornado para o log de startup do FastAPI

async def create_task(task_id: str, description: str):
    async with aiosqlite.connect(SessionManager.get().db_path) as db:
        await db.execute(
            "INSERT INTO tasks (id, description, status) VALUES (?, ?, 'running')
            (task_id, description)
        )
        await db.commit()

async def finish_task(task_id: str, status: str = "done"):
    async with aiosqlite.connect(SessionManager.get().db_path) as db:
        await db.execute(
            "UPDATE tasks SET status = ?, finished_at = datetime('now', 'localtim

```

```

        (status, task_id)
    )
    await db.commit()

async def save_log(task_id: str, level: str, message: str):
    async with aiosqlite.connect(SessionManager.get().db_path) as db:
        await db.execute(
            "INSERT INTO logs (task_id, level, message) VALUES (?, ?, ?)",
            (task_id, level, message)
        )
        await db.commit()

async def save_generated_file(task_id: str, filename: str, size_bytes: int):
    async with aiosqlite.connect(SessionManager.get().db_path) as db:
        await db.execute(
            "INSERT INTO generated_files (task_id, filename, size_bytes) VALUES (
            (task_id, filename, size_bytes)
            )
        )
        await db.commit()

async def get_task_logs(task_id: str) -> list:
    async with aiosqlite.connect(SessionManager.get().db_path) as db:
        db.row_factory = aiosqlite.Row
        async with db.execute(
            "SELECT level, message, created_at FROM logs WHERE task_id = ? ORDER
            (task_id,)"
        ) as cursor:
            return [dict(row) async for row in cursor]

async def list_all_sessions() -> list:
    """
    Varre todas as pastas ses_N no HD de backup e retorna metadados de cada sessão
    Útil para um painel de histórico futuro.
    """
    import sqlite3
    sm = SessionManager.get()
    sessions = []

    if not sm.vortex_root or not sm.vortex_root.exists():
        return []

    for folder in sorted(sm.vortex_root.iterdir()):
        if not folder.is_dir() or not re.match(r"^ses_\d+$", folder.name):

```

```

        continue
db_files = list(folder.glob("*.db")) + list(folder.glob("*.sqlite"))
if not db_files:
    continue
db_file = db_files[0]
try:
    conn = sqlite3.connect(str(db_file))
    meta_rows = conn.execute("SELECT key, value FROM session_meta").fetch
    meta = dict(meta_rows)
    task_count = conn.execute("SELECT COUNT(*) FROM tasks").fetchone()[0]
    conn.close()
    sessions.append({
        "folder":      folder.name,
        "db_path":      str(db_file),
        "session_number": meta.get("session_number"),
        "started_at":    meta.get("started_at"),
        "task_count":    task_count,
    })
except Exception:
    sessions.append({"folder": folder.name, "db_path": str(db_file), "err

return sessions

```

Integração com o `main.py` — Log de Startup

```

# backend/main.py — adicione dentro do lifespan
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Inicializa o banco no HD de backup e loga o resultado
    init_result = await init_db()

    if "warning" in init_result:
        print(f"⚠️ AVISO DE BANCO: {init_result['warning']}")
    else:
        print(f"✅ HD de backup encontrado: {init_result.get('backup_folder_found')}")

    print(f"📁 Vortex: {init_result.get('vortex_root')}")
    print(f"📁 Sessão #{init_result.get('session_number')} iniciada")
    print(f"📁 Banco desta sessão: {init_result.get('session_db_path')}")

    yield

```

Exemplo Real de Como Fica no HD

Suponha que seu HD de backup está montado em `/media/alvaro/BackupHD` e a pasta de bancos se chama `databases/`:

```
/media/alvaro/BackupHD/databases/
├─ projeto_a.db           ← banco pré-existente (intocado)
├─ projeto_b.sqlite       ← banco pré-existente (intocado)
└─ Vortex/                ← criada pelo agente
    ├─ ses_1/              ← 1ª execução (ex: 10/06/2025 14:30)
    │   └─ session.sqlite
    ├─ ses_2/              ← 2ª execução (ex: 11/06/2025 09:15)
    │   └─ session.sqlite
    └─ ses_3/              ← 3ª execução (em andamento)
        └─ session.sqlite
```

Extensão detectada: como `projeto_b.sqlite` existe, o sistema detecta `.sqlite` como padrão e cria `session.sqlite` dentro de cada `ses_N/`. Se os bancos pré-existent usassem `.db`, criaria `session.db`.

Novo Endpoint — Histórico de Sessões

```
# backend/api/tasks.py — adicione esta rota
from database import list_all_sessions

@router.get("/sessions")
async def get_all_sessions():
    """Lista todas as sessões gravadas no HD de backup."""
    sessions = await list_all_sessions()
    return {"sessions": sessions, "total": len(sessions)}
```

Comandos Úteis para Inspeccionar o HD de Backup

```
# Ver onde o HD está montado
lsblk -o NAME,MOUNTPOINT,LABEL,SIZE
# ou
df -h | grep media

# Ver a estrutura Vortex
ls -la /media/$USER/*/databases/Vortex/
```

```
# Ver banco de uma sessão específica
sqlite3 /media/$USER/HD_BACKUP/databases/Vortex/ses_1/session.sqlite \
    "SELECT * FROM session_meta;"

# Ver todas as tarefas de uma sessão
sqlite3 /media/$USER/HD_BACKUP/databases/Vortex/ses_3/session.sqlite \
    "SELECT id, description, status, created_at FROM tasks;"

# Ver logs de uma tarefa específica
sqlite3 /media/$USER/HD_BACKUP/databases/Vortex/ses_3/session.sqlite \
    "SELECT level, message, created_at FROM logs WHERE task_id='abc123' ORDER BY id"

# Contar sessões gravadas
ls /media/$USER/HD_BACKUP/databases/Vortex/ | wc -l
```

14. Rodando como Serviço do Sistema (systemd)

Esta é uma das maiores vantagens do Linux Mint: os serviços **iniciam automaticamente com o sistema**, sobrevivem a crashes, têm logs centralizados com `journalctl`, e são gerenciados com um único comando.

systemd/manus-agent.service

```
[Unit]
Description=Manus Local Agent — FastAPI Backend
After=network.target
Wants=network.target

[Service]
Type=simple
User=SEU_USUARIO
WorkingDirectory=/home/SEU_USUARIO/manus-local/backend

# Carrega variáveis do .env
EnvironmentFile=/home/SEU_USUARIO/manus-local/.env

# Necessário para MSS e PyAutoGUI acessarem o display
Environment=DISPLAY=:0
Environment=XAUTHORITY=/home/SEU_USUARIO/.Xauthority

ExecStart=/home/SEU_USUARIO/manus-local/venv/bin/uvicorn main:app \
    --host 127.0.0.1 \
    --port 8000 \
```



```

--workers 1

# Reinicia automaticamente se cair
Restart=always
RestartSec=5

# Logs via journalctl
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

```

systemd/manus-tunnel.service

```

[Unit]
Description=Cloudflare Tunnel - Manus Agent
After=network.target manus-agent.service
Requires=network.target

[Service]
Type=simple
User=SEU_USUARIO

ExecStart=/usr/bin/cloudflared tunnel \
    --config /home/SEU_USUARIO/manus-local/cloudflared-config.yml \
    run

Restart=always
RestartSec=10

StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

```

Instalando os serviços

```

# Substitua SEU_USUARIO pelo seu nome de usuário real
USER=$(whoami)

# Copia os arquivos de serviço
sudo cp ~/manus-local/systemd/manus-agent.service /etc/systemd/system/

```

```
sudo cp ~/manus-local/systemd/manus-tunnel.service /etc/systemd/system/

# Substitui SEU_USUARIO pelo nome real nos arquivos
sudo sed -i "s/SEU_USUARIO/$USER/g" /etc/systemd/system/manus-agent.service
sudo sed -i "s/SEU_USUARIO/$USER/g" /etc/systemd/system/manus-tunnel.service

# Recarrega o systemd
sudo systemctl daemon-reload

# Habilita os serviços (inicia junto com o boot)
sudo systemctl enable manus-agent
sudo systemctl enable manus-tunnel

# Inicia agora
sudo systemctl start manus-agent
sudo systemctl start manus-tunnel

# Verifica se estão rodando
sudo systemctl status manus-agent
sudo systemctl status manus-tunnel
```

15. Fluxo Completo de Execução

```
Usuário digita: "Acesse o G1 e me diga as 5 principais notícias de hoje, salve em
|
▼
POST /api/tasks/ { description: "..."} [JWT no header]
|
▼
FastAPI cria task_id, inicia AgentOrchestrator em asyncio background task
|
▼
Frontend abre WebSocket: wss://agente.seudominio.com/ws/{task_id}?token=...
|
▼
== ITERAÇÃO 1 ==
DeepSeek recebe a tarefa, responde:
{
  "action": "browser_navigate",
  "description": "Vou acessar o G1 para buscar as notícias",
  "params": { "url": "https://g1.globo.com" }
}
|
▼
```

Playwright abre/controla o Google Chrome já instalado no Linux Mint

Log via WS: "🔧 browser_navigate: Vou acessar o G1..."

Screenshot capturado automaticamente e enviado ao frontend



== ITERAÇÃO 2 ==

DeepSeek recebe: { "url": "https://g1.globo.com", "title": "G1 - O Portal de Notí

DeepSeek responde:

```
{
  "action": "browser_extract_text",
  "description": "Vou extrair o texto da página para identificar as notícias"
}
```



Playwright extrai texto da página → retorna 6000 chars



== ITERAÇÃO 3 ==

DeepSeek analisa o texto, identifica as 5 notícias,
responde:

```
{
  "action": "file_write",
  "description": "Salvando as notícias encontradas em arquivo .txt",
  "params": {
    "filename": "noticias_g1_hoje.txt",
    "content": "Notícias G1 - [data]\n\n1. Título...\n2. Título..."
  }
}
```



FileManager salva em ~/manus-workspace/noticias_g1_hoje.txt

Log via WS: "✅ Arquivo salvo"

Frontend atualiza lista de arquivos com link de download



== ITERAÇÃO 4 ==

```
{
  "action": "finish",
  "result": "Extraí as 5 principais notícias do G1 e salvei em noticias_g1_hoje.t"
}
```



Status: DONE ✅

16. Guia de Instalação Passo a Passo — Linux Mint

Passo 1 — Dependências do Sistema

```
# Atualiza o sistema
sudo apt update && sudo apt upgrade -y

# Python 3.11+ (Mint 21+ já vem com 3.10/3.11)
sudo apt install -y python3 python3-pip python3-venv python3-dev

# Verifica versão
python3 --version    # deve ser 3.10 ou 3.11+

# Dependências para MSS e PyAutoGUI no X11
sudo apt install -y \
    python3-xlib \
    scrot \
    xdotool \
    libx11-dev \
    libxtst-dev \
    libpng-dev

# Node.js 18+ (para buildar o frontend)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Verifica
node --version    # deve ser 20+
npm --version

# Git (provavelmente já instalado)
sudo apt install -y git
```

Passo 2 — Clone e Configuração

```
# Cria e acessa a pasta do projeto
cd ~
git clone https://github.com/seu-usuario/manus-local
cd manus-local

# Cria o ambiente virtual Python
python3 -m venv venv

# Ativa o ambiente virtual
```

```
source venv/bin/activate

# Instala as dependências Python
pip install -r backend/requirements.txt

# Instala as dependências do Playwright para Linux
# IMPORTANTE: não precisa instalar Chromium pois usaremos o Chrome do sistema
pip install playwright
playwright install-deps chromium # instala apenas as dependências do sistema

# Cria a workspace
mkdir -p ~/manus-workspace

# Copia o .env de exemplo
cp .env.example .env
```

backend/requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
pydantic==2.8.2
pydantic-settings==2.4.0
httpx==0.27.2
playwright==1.47.0
mss==9.0.2
Pillow==10.4.0
aiofiles==24.1.0
aiosqlite==0.20.0
python-jose[cryptography]==3.3.0
python-multipart==0.0.9
slowapi==0.1.9
pyautogui==0.9.54
python-xlib==0.33
opencv-python-headless==4.10.0.84
```

Nota: `opencv-python-headless` no lugar de `opencv-python` — a versão headless não tenta abrir janelas de display, o que é mais adequado para um serviço em background.

Passo 3 — Build do Frontend

```
cd ~/manus-local/frontend

npm install
```

```
# Builda o React para produção (gera a pasta dist/)
npm run build

# Volta para o backend
cd ~/manus-local/backend
```

Passo 4 — Configurar o .env

```
# Abre o .env no editor padrão do Mint (gedit, xed, ou nano)
nano ~/manus-local/.env

# Preencha os valores (veja a seção 17)
```

Passo 5 — Teste Manual

```
# Ativa o venv
source ~/manus-local/venv/bin/activate

# Inicia o FastAPI em modo desenvolvimento
cd ~/manus-local/backend
uvicorn main:app --host 127.0.0.1 --port 8000 --reload

# Em outro terminal: testa se está funcionando
curl http://localhost:8000/docs

# Se abrir a documentação automática do FastAPI, está funcionando
```

Passo 6 — Instalar como Serviço

```
# Siga as instruções da seção 14 acima
sudo systemctl start manus-agent

# Verifica logs em tempo real
sudo journalctl -u manus-agent -f
```

Passo 7 — Cloudflare Tunnel

```
# Instala o cloudflared (veja seção 12)
# Configura o tunnel (veja seção 12)
sudo systemctl start manus-tunnel

# Verifica
sudo journalctl -u manus-tunnel -f
```

17. Configuração do Ambiente (.env)

```
# =====
# SEGURANÇA
# =====
# Gere com: openssl rand -hex 32
SECRET_KEY=cole_aqui_o_resultado_de_openssl_rand_hex_32

ADMIN_USER=admin
ADMIN_PASS=sua_senha_forte_aqui_min_12_caracteres

# =====
# APIs DE IA
# =====
DEEPSEEK_API_KEY=sk-...

# Provedor de visão: openai ou google
VISION_PROVIDER=openai

# Preencha conforme o provider:
OPENAI_API_KEY=sk-...      # se VISION_PROVIDER=openai
GOOGLE_API_KEY=AIza...     # se VISION_PROVIDER=google

# =====
# CONFIGURAÇÕES DO AGENTE
# =====
MAX_ITERATIONS=30
WORKSPACE_PATH=/home/SEU_USUARIO/manus-workspace

# =====
# GOOGLE CHROME - Linux Mint
# =====
# Caminho padrão do Chrome no Linux Mint:
CHROME_BINARY=/usr/bin/google-chrome

# Se você instalou como google-chrome-stable:
# CHROME_BINARY=/usr/bin/google-chrome-stable

# Porta CDP do Chrome (não mude sem necessidade)
CHROME_DEBUG_PORT=9222

# =====
# CLOUDFLARE
```

```
# =====  
EXTERNAL_DOMAIN=agente.seudominio.com
```

18. Comandos do Dia a Dia

No Linux Mint com systemd, a gestão do agente é simples:

```
# === VERIFICAR STATUS ===  
sudo systemctl status manus-agent  
sudo systemctl status manus-tunnel  
  
# === VER LOGS EM TEMPO REAL ===  
sudo journalctl -u manus-agent -f          # logs do FastAPI  
sudo journalctl -u manus-tunnel -f         # logs do cloudflared  
sudo journalctl -u manus-agent -n 100      # últimas 100 linhas  
  
# === CONTROLAR OS SERVIÇOS ===  
sudo systemctl start  manus-agent  
sudo systemctl stop   manus-agent  
sudo systemctl restart manus-agent  
  
# === APÓS MUDAR O CÓDIGO ===  
cd ~/manus-local/backend  
source ~/manus-local/venv/bin/activate  
# (testa localmente se quiser)  
sudo systemctl restart manus-agent  
  
# === APÓS MUDAR O FRONTEND ===  
cd ~/manus-local/frontend  
npm run build  
sudo systemctl restart manus-agent      # FastAPI serve a nova versão  
  
# === VERIFICAR WORKSPACE ===  
ls -lah ~/manus-workspace/  
  
# === VER BANCO DE DADOS ===  
sqlite3 ~/manus-local/agent.db "SELECT * FROM tasks ORDER BY created_at DESC LIMIT 100"  
  
# === ATUALIZAR DEPENDÊNCIAS PYTHON ===  
source ~/manus-local/venv/bin/activate  
pip install -r ~/manus-local/backend/requirements.txt --upgrade  
  
# === CHECAR SE A PORTA ESTÁ OCUPADA ===  
ss -tlnp | grep 8000
```



```
ss -tlnp | grep 9222    # porta CDP do Chrome
```

19. Roadmap de Implementação

Fase 1 — MVP Funcional (1-2 semanas)

- Instalação das dependências no Linux Mint
- Backend FastAPI rodando localmente (`localhost:8000`)
- Orquestrador com DeepSeek + loop básico
- Ferramenta `browser_navigate` com Chrome via CDP
- WebSocket básico para logs
- Frontend mínimo (chat + logs)
- Autenticação JWT

Fase 2 — Ferramentas Completas (1 semana)

- Todas as ferramentas do browser (click, type, extract, scroll)
- ShellTool com whitelist bash
- FileManager com workspace isolada
- ScreenshotTool via MSS/X11
- VisionTool plugável (OpenAI ou Google)
- Botões de pausar/parar/confirmar no frontend

Fase 3 — Produção (3-5 dias)

- Cloudflare Tunnel instalado e testado
- Serviços systemd configurados e habilitados
- Frontend buildado e servido pelo FastAPI
- Testes end-to-end com uma tarefa real
- Cloudflare Access ativado (opcional mas recomendado)

Fase 4 — Melhorias (contínuo)

- Memória persistente entre tarefas (RAG simples com SQLite + embeddings)
- Upload de arquivos pelo usuário para o agente processar
- Dashboard de histórico de tarefas com replay de logs
- Múltiplas tarefas em fila
- Notificação via email ou Telegram quando tarefa conclui
- OpenCV para localização visual de elementos por imagem (template matching)
- Integração com ferramentas Linux específicas: `pdftotext`, `ffmpeg`, `imagemagick`

20. Limitações e Riscos

Limitações Conhecidas

Limitação	Impacto	Mitigação
Chrome via CDP pode ser instável se fechado manualmente	Agente perde o navegador	Auto-relançar Chrome na próxima chamada (já implementado)
MSS requer sessão X11 ativa	Screenshots falham se não há display	Setar <code>DISPLAY=:0</code> no serviço systemd (já configurado)
DeepSeek com latência variável	Loop mais lento às vezes	Timeout de 90s por chamada
SQLite sem suporte a múltiplos escritores concorrentes	Problema se rodar 2 tarefas ao mesmo tempo	Adequado para uso solo; use PostgreSQL se precisar escalar
PyAutoGUI FAILSAFE move o mouse para o canto	Pode cancelar ações acidentalmente	Desabilitar FAILSAFE com cuidado se necessário

Riscos de Segurança e Mitigações

Risco	Mitigação
Acesso não autorizado	JWT (24h) + Cloudflare Access (opcional)
Agente executar <code>rm -rf</code> ou <code>sudo</code>	Whitelist + verificação de padrões bloqueados

Path traversal no FileManager	<code>Path.resolve()</code> + verificação de prefix
Loop infinito custando muito na API	<code>max_iterations = 30</code> rígido
<code>.env</code> com API keys vazar	<code>chmod 600</code> , no <code>.gitignore</code> , nunca <code>commitar</code>
Porta 9222 (Chrome CDP) exposta	Apenas <code>127.0.0.1</code> , nunca <code>0.0.0.0</code>
Cloudflare Tunnel expõe o servidor	HTTPS automático, autenticação JWT obrigatória

Custos Estimados de API

API	Custo por tarefa (média 20 iterações)
DeepSeek Chat	~\$0.008 - \$0.04
GPT-4o Mini Vision (10 análises)	~\$0.01 - \$0.03
Total por tarefa	~\$0.02 - \$0.07

Para uso moderado (10-20 tarefas/dia), custo mensal estimado: **\$6 - \$42/mês**.

Resumo Executivo

Componente	Tecnologia	Justificativa para Linux Mint
Sistema Operacional	Linux Mint	Leve (~1 GB RAM), bash nativo, systemd, sem licença
Backend	Python + FastAPI	Async nativo, zero friction no Linux
Frontend	React + Vite	Build estático, sem servidor Node em produção
WebSocket	FastAPI nativo	Sem dependência extra
Navegador	Google Chrome (já instalado)	Sem baixar nada extra, CDP funciona perfeitamente
Automação web	Playwright + CDP	Conecta ao Chrome existente, mais estável
Automação desktop	PyAutoGUI + Xlib	Funciona nativamente no X11 do Mint

Screenshot	MSS + Pillow	X11 nativo, sem configuração
Shell	bash + whitelist	Mais poderoso e previsível que cmd/PowerShell
IA Principal	DeepSeek API	Custo baixo, JSON forçado, raciocínio excelente
IA Visão	Plugável (OpenAI/Google)	Troca de provider em 1 linha no .env
Banco	SQLite +aiosqlite	Zero configuração, perfeito para uso solo
Serviços	systemd	Boot automático, restart automático, logs centralizados
Túnel	Cloudflare Tunnel	Sem IP fixo, HTTPS automático, gratuito
Autenticação	JWT + Cloudflare Access	Dupla camada, sem abrir portas no roteador

Versão 2.0 — Adaptado para Linux Mint com Google Chrome. Hardware: i5 antigo + 16 GB DDR. Toda inteligência delegada para APIs externas. O Linux Mint é a escolha ideal: consome menos RAM que Windows, gerencia serviços com systemd, e o bash torna todos os scripts mais simples e previsíveis.